Computational Physics 2 Classes No.2 - Report
2D Laplace Equation solved with the Finite Element Method

Natalia Kowalczyk

April 18, 2026

1 Introduction

In this report, the two-dimensional Laplace/Poisson problem with a point charge located outside the computational domain is solved numerically with the finite element method. The exact logarithmic potential is used to impose boundary conditions and to compare the numerical solution with the analytical result.

Two types of basis functions are considered: bilinear and biparabolic. For both cases, the local stiffness matrix is constructed, the global system is assembled, and the accuracy of the FEM solution is examined by comparing the potential and the numerical error for different mesh densities.

2 Results

The calculations were performed for the computational domain $[0, L] \times [0, L]$ with $L = 5$ and with the source located at $\mathbf{r}_c = (-1, 0)$. For the reference case, the mesh parameter was set to $N = 4$, and additional calculations were carried out for several values of N in order to study the convergence of the method.

D.I.1 Mesh generation and local-to-global mapping

For the bilinear basis and for the reference case $N = 4$, the computational domain $[0, 5] \times [0, 5]$ was divided into $4 \times 4 = 16$ square elements. Since bilinear elements use only corner nodes, the resulting mesh contains $(N + 1)^2 = 25$ global nodes, which is consistent with the finite element construction used in this task. The node coordinates and the local-to-global mapping table $nlg(k, i)$ were generated and saved to the output file `bilinear_N4.txt`. This table identifies, for each element k and each local node index $i = 1, \ldots, 4$, the corresponding global node number together with its physical coordinates.

The generated mesh is regular and uniform, with spacing $a = L/N = 1.25$ in both spatial directions. The nodes are distributed on a Cartesian grid, starting from $(0, 0)$ and ending at $(5, 5)$. This confirms that the mesh generation procedure and the node indexing were implemented correctly and provide the necessary data for global matrix assembly.

D.I.2 Local stiffness matrix for the bilinear basis

Before imposing boundary conditions, the local stiffness matrix for the bilinear element was computed using Gauss quadrature. The obtained matrix was

$$E = \begin{bmatrix} 0.666666666667 & -0.166666666667 & -0.166666666667 & -0.333333333333 \\ -0.166666666667 & 0.666666666667 & -0.333333333333 & -0.166666666667 \\ -0.166666666667 & -0.333333333333 & 0.666666666667 & -0.166666666667 \\ -0.333333333333 & -0.166666666667 & -0.166666666667 & 0.666666666667 \end{bmatrix}.$$

The matrix is symmetric, which is expected for the Laplace operator and confirms the correctness of the numerical integration and the derivatives of the shape functions. The positive diagonal terms and negative off-diagonal couplings reflect the standard structure of the stiffness matrix for a four-node bilinear finite element. The matrix was also written to the file `local_stiffness_matrix.txt` for verification and further use in the global assembly procedure.

D.I.3 Node comparison for the bilinear basis

Figure 1 shows the potential values in the nodes obtained from the FEM solution and from the exact analytical formula. The two curves are almost indistinguishable, which confirms that the numerical approximation is accurate at the nodal level.

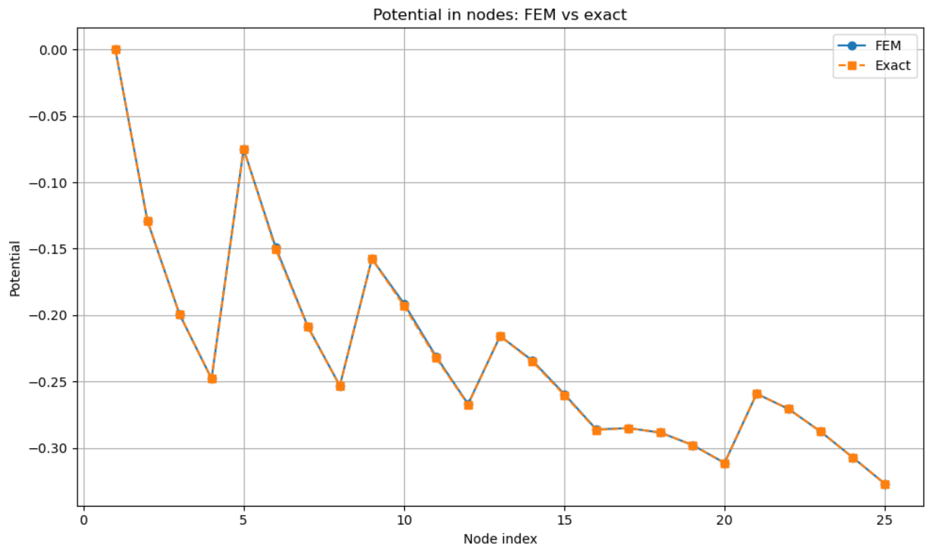

Figure 1: Potential in the nodes for the bilinear basis: FEM solution and exact solution.

Figure 2 presents the difference between the FEM and exact solution in the nodes. As expected, the error is exactly zero on the boundary nodes, while small nonzero deviations appear in the interior of the domain.

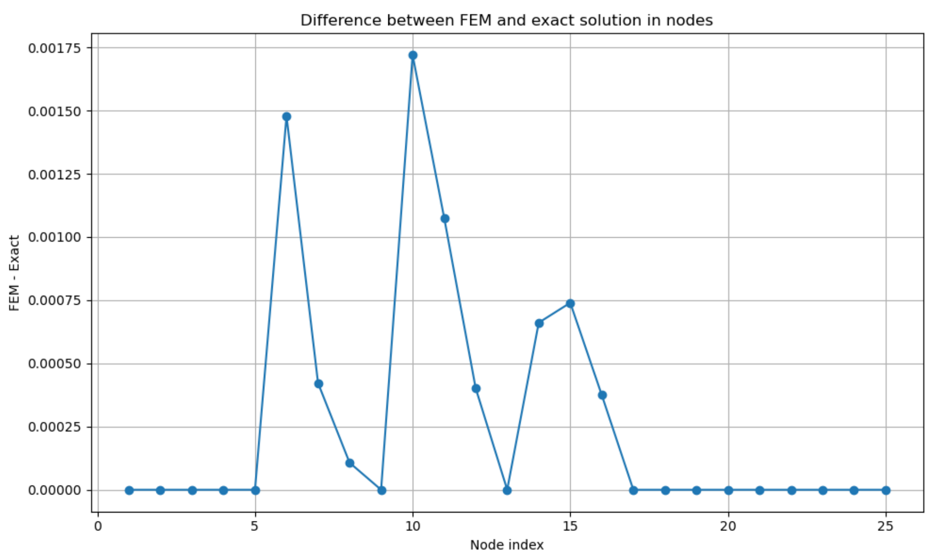

Figure 2: Difference between the FEM and exact nodal values for the bilinear basis.

D.I.4 Scan of the bilinear solution inside the elements

To examine the solution inside the elements, the local coordinates were scanned from -1 to 1 with step 0.2, and the physical coordinates were reconstructed using the bilinear mapping. Figure 3 shows the FEM potential in the full computational domain. The potential becomes more negative as the distance from the external source configuration increases, and the field varies smoothly across element boundaries.

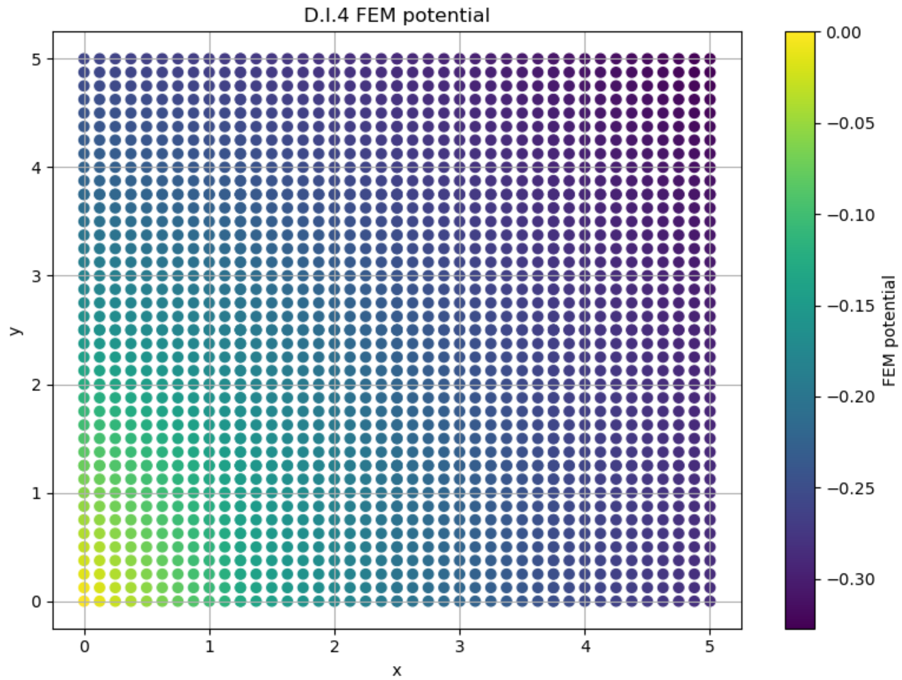

Figure 3: Bilinear FEM potential obtained from the element scan.

Figure 4 shows the pointwise difference between the FEM and exact solution. The error is small throughout the domain, but it is visibly larger in the interior than on the boundary, which is consistent with the fact that the exact boundary values were imposed directly.

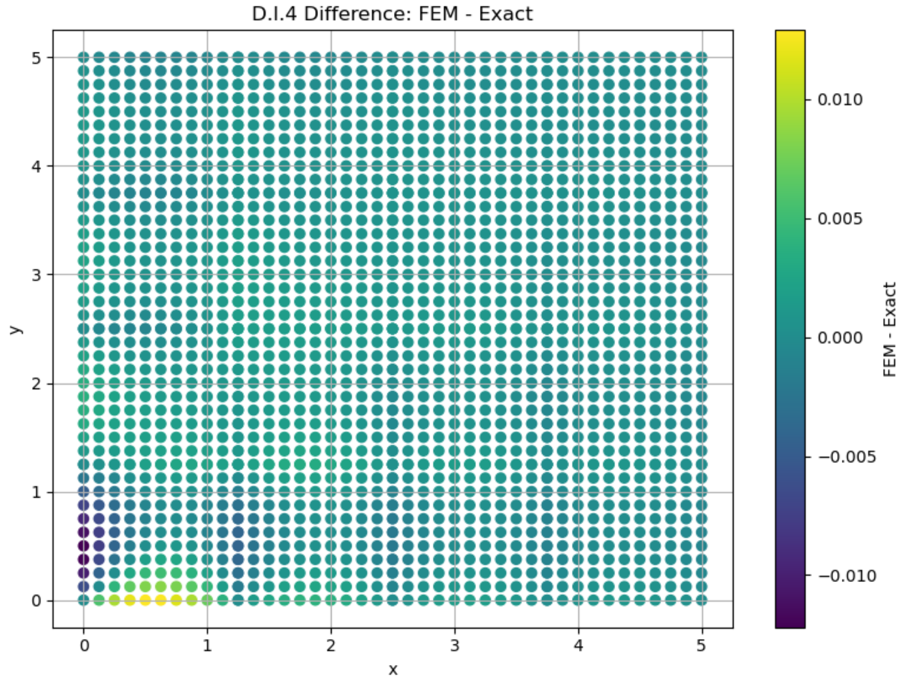

Figure 4: Difference between the bilinear FEM solution and the exact solution in the scanned element points.

D.I.5 Maximum error for the bilinear basis

The maximal absolute difference between the FEM and exact solution was computed for several mesh densities. The obtained values were:

$$
\begin{aligned}
N &= 2 : 0.030564432493, \\
N &= 3 : 0.018858893502, \\
N &= 4 : 0.012906355241, \\
N &= 5 : 0.010252671281, \\
N &= 6 : 0.008401905202, \\
N &= 8 : 0.005705658422, \\
N &= 10 : 0.004054245685.
\end{aligned}
$$

These results are plotted in Figure 5. The maximum error decreases systematically with increasing N, which confirms convergence of the bilinear finite element approximation.

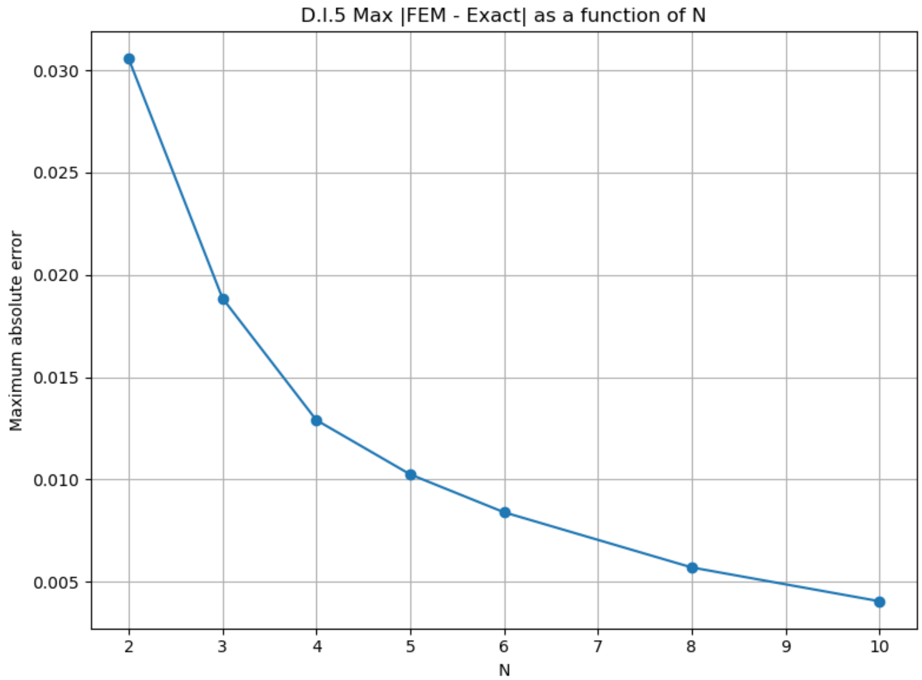

Figure 5: Maximum absolute error as a function of N for the bilinear basis.

D.II.1 Scan of the biparabolic solution inside the elements

The same element scan was repeated for the biparabolic basis. Figure 6 shows the FEM potential reconstructed from the 9-node approximation. The solution is smooth and follows the same physical trend as in the bilinear case, but the higher-order interpolation gives a visibly more accurate representation inside the elements.

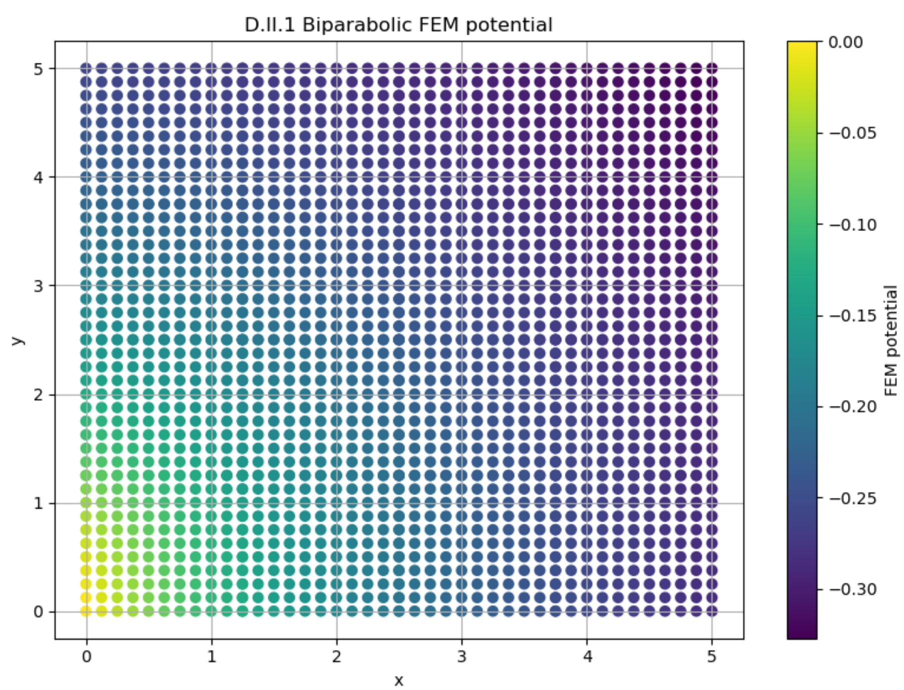

Figure 6: Biparabolic FEM potential obtained from the element scan.

Figure 7 presents the difference between the biparabolic FEM solution and the exact potential. Compared with the bilinear case, the error is clearly smaller, which shows the advantage of the higher-order basis.

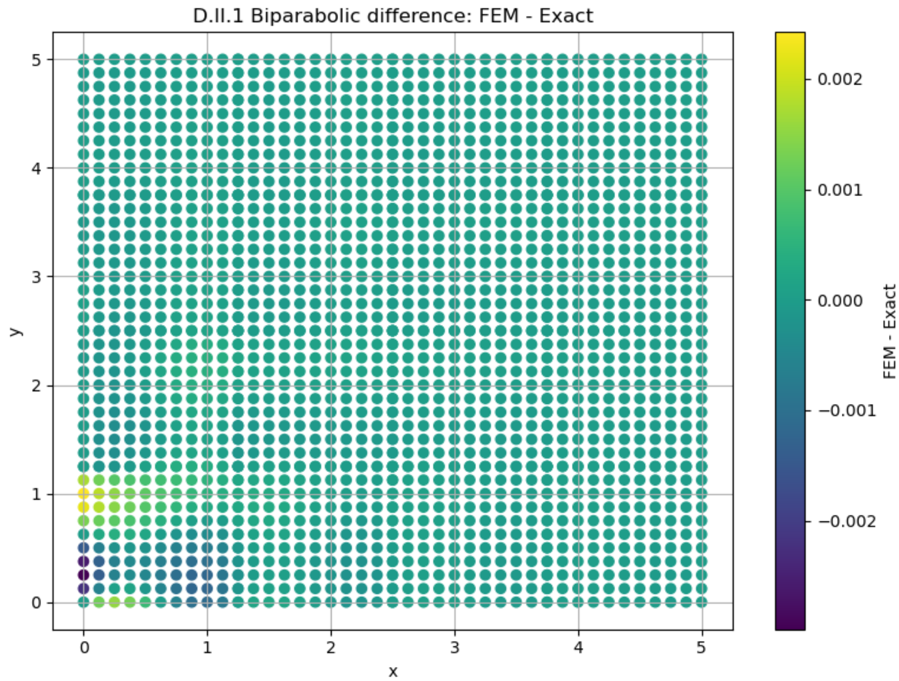

Figure 7: Difference between the biparabolic FEM solution and the exact solution in the scanned element points.

D.II.2 Maximum error for the biparabolic basis

The maximal absolute error for the biparabolic basis was also evaluated for several values of N:

$$\begin{aligned} N = 2 &: 0.011249518036, \\ N = 3 &: 0.005781329021, \\ N = 4 &: 0.002984113813, \\ N = 5 &: 0.001624454558, \\ N = 6 &: 0.000966050607, \\ N = 8 &: 0.000432967035, \\ N = 10 &: 0.000239337569. \end{aligned}$$

These results are shown in Figure 8. The error decreases much faster than for the bilinear basis, and for every tested mesh density the biparabolic approximation is significantly more accurate.

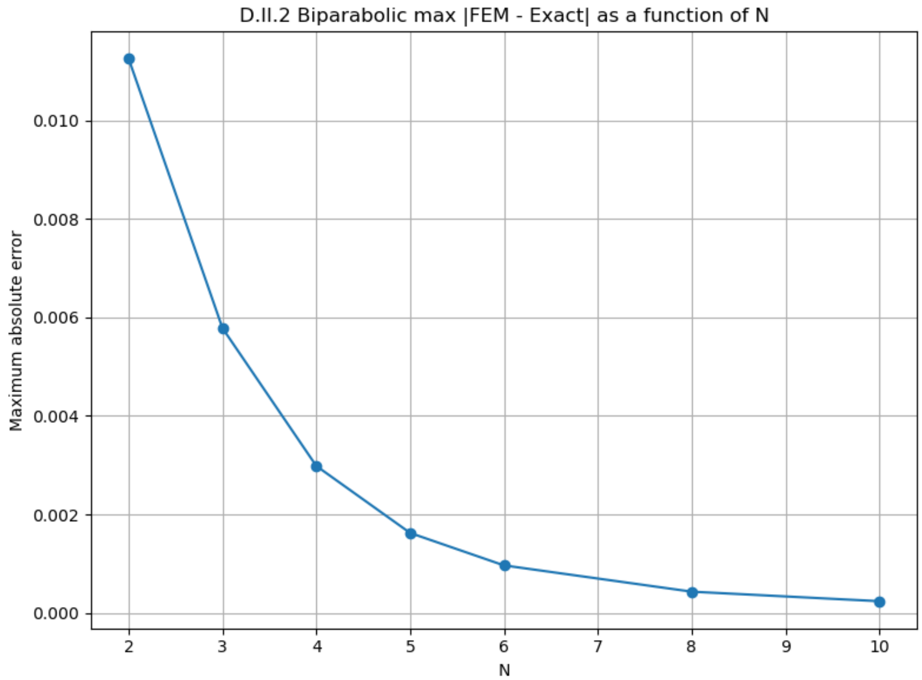

Figure 8: Maximum absolute error as a function of N for the biparabolic basis.

Comparison of bilinear and biparabolic bases

A direct comparison of the maximum errors shows that the biparabolic basis gives a clear improvement over the bilinear one. For example, for $N = 4$ the bilinear approximation gives a maximum error of 0.012906355241, while the biparabolic approximation gives only 0.002984113813. For $N = 10$, the corresponding errors are 0.004054245685 and 0.000239337569. This confirms that the higher-order basis captures the variation of the solution much more efficiently and leads to substantially better accuracy on the same computational domain.

Overall, the obtained results are fully consistent with the expected behavior of the finite element method. Both approximations converge as the mesh is refined, but the biparabolic basis converges faster and reproduces the exact logarithmic potential more accurately than the bilinear basis.

3 Conclusions

The finite element method reproduced the exact logarithmic potential well for both types of basis functions. In both cases, the numerical error decreased with increasing mesh density, which confirms convergence of the method.

The biparabolic basis gave significantly smaller errors than the bilinear basis for all tested values of N. Therefore, the higher-order approximation provides a more accurate solution on the same computational domain.